职业院校技能大赛：演示代码精选 (智慧康养健康监测与预警系统)

💡 展示核心逻辑

本次展示的核心目标是向评委证明：项目不仅具备**工业级的物联网接入能力**，更是融合了**深度学习边缘推理**、**高频数据可视化**与**可编排智能体 (Agent)** 的前沿性系统工程。

---

1. 成员一（队长）：医疗级张量推理与边缘融合

展示重点：强调在后端推理层面，系统不再依赖单一规则，而是利用 `PyTorch` 实现了支持硬件加速的多模态张量推理，并结合数学融合框架保证预测鲁棒性。

- **[核心代码] 多模态健康张量推断与分数融合 (`backend/ml/inference.py`)**

```
# 核心架构：深度张量网络前向推断 (Tensor Forward Inference) 与概率校准
def predict(self, payload):
    # 1. 结构化特征提取空间，执行特征工程预处理与高维数据缩放变换
    feature_frame = build_single_feature_frame(payload).loc[:,
artifacts.feature_columns]
    scaled_features = artifacts.scaler.transform(feature_frame)

    # 2. 构建计算图：启用无梯度计算图 (No-Grad) 下的并行推断，支持 CUDA 硬件底层加速
    tensor = torch.tensor(scaled_features, dtype=torch.float32,
device=self.device)
    with torch.no_grad():
        outputs = artifacts.model(tensor)
        # 激活与降维：通过 Sigmoid 将网络对数优势 (Logits) 映射为 [0, 1] 概率分布
        risk_prob = float(torch.sigmoid(outputs["risk_score"]).item())
        hr_prob = float(torch.sigmoid(outputs["hr_alert"]).item())

    # 3. 证据融合 (Dempster-Shafer 启发)：动态加权融合模型概率分布与医学临床阈值
    final_score = fuse_health_scores(rule_baseline, risk_raw_to_score(risk_prob))
    return {"health_score": final_score, "hr_probability": hr_prob}
```

---

2. 成员二（数据通路）：物联网底层嗅探与协议自适应

展示重点：向评委展示团队具备对硬件协议栈直接编程的能力。我们实现了单网关节点对异构数据流的时分多址 (TDMA) 级信道调度。

- **[核心代码] 硬件级动态协议跳频与信道复用调度 (`iot/serial_reader.py`)**

```python
# 核心架构：时分多址 (TDMA) 信道复用与紧急报文硬件级优先抢占
def run_collection_cycle(self, connection, active_packet_type):
    # 利用硬件单调时钟 (Monotonic Time) 避免时间漂移
    elapsed = time.monotonic() - self.cycle_started_at
    target_packet_type = 5 # 默认波特率：通用低功耗健康广播信道 (BLE Beacon)

    # 动态帧插入：强制中断轮询，阶段性切换至 SOS 专用应急响应信道
    if active_packet_type == 5 and elapsed >= self.response_cycle_seconds:
        target_packet_type = 4 # 激活高频报文嗅探 (Fast-Scan protocol)

    # 越过驱动层直接向硬件下发 AT 控制指令，毫秒级完成频段与协议跳变
    if target_packet_type != active_packet_type:
        self.switch_packet_type(connection, packet_type=target_packet_type)
        return target_packet_type
```

---

3. 成员三（前端开发）：高频实时流直连与状态融合

展示重点：证明前端架构不仅是静态界面的渲染，更是涉及到处理底层 WebSocket 双工通信、维护**高频数据流 (Reactive Streaming)** 以及在内存中进行状态切片融合的系统级能力。

- **[核心代码] 响应式滑动窗口与大并发长连接流处理 (`useDeviceTrend.ts`)**

```cpp
// 核心架构：前端数据流 (Reactive Streaming) 引擎与 WebSocket 状态同构
function connectHealthSocket(mac) {
    // 建立基于 WebSocket 的底层全双工管道，彻底打破 HTTP 短轮询的物理延迟
    healthSocket = api.healthSocket(mac);

    // 注册高频微任务回调：接管硬件级毫秒数据流推送 (Server-to-Client Push)
    healthSocket.onmessage = (event) => {
        const sample = JSON.parse(event.data);

        // 状态切片无缝融合 (Stateful Slice Merge)：将离散的数据帧平滑织入全局快照
        const mergedLatest = mergeHealthSample(options.latest.value[mac], sample);
        options.latest.value = { ...options.latest.value, [mac]: mergedLatest };

        // 基于时间滑动窗口 (Sliding Window O(1)) 维护内存追踪数组
        // 自动出队陈旧数据，防止高频通信导致浏览器内存泄漏 (Memory Leak)
        const previous = trendStore.value[mac] ?? [];
        const mergedSeries = mergeHealthSeries([...previous, mergedLatest]);
        trendStore.value = { ...trendStore.value, [mac]: mergedSeries.slice(-240)
};
    };
}
```

---

4. 成员四（智能体）：状态机流转与反应式编排

展示重点：展现生成式 AI 在系统中是如何被工程化落地。通过图结构（Graph）调度和流式引擎，让 Agent 的大模型调用“可控”、“黑盒清晰可见”。

- **[核心代码] 反应式大模型执行链 (Reactive Graph) 调度引擎 (`langgraph_health_agent.py`)**

```
# 核心架构：基于有向无环图 (DAG) 与状态自包含的代理编排 (Agent Orchestration)
def _stream_execute(self, state):
    # 动态装配 Agent 思维链阶段 (Chain-of-Thought Stages) 的可路由图节点
    stages = [
        ("route",     "算力路由", self._route_node),    # 感知层：多路模型
(Cloud/Local)意图路由分发
        ("retrieve", "知识增强", self._retrieve_node),# 认知层：RAG (检索增强生成)
向量数据库召回
        ("generate", "回复生成", self._generate_node) # 输出层：LLM 的流式输出
(Streaming Payload)
    ]

    # 异步生成器 (Async Generator) 结合状态机流转设计，实时暴露出模型的内部心智
    for stage_key, _, handler in stages:
        state.update(handler(state)) # 状态变迁计算 (State Transition)
        # Yield Server-Sent Events 流式事件，实时驱动前端控制台 (Dashboard) 同步更新
        yield {"type": "trace.node_completed", "stage": stage_key, "context":
state}
```

🚩 演示技巧建议

1. **关键词重读（重要）**：在 PPT 中加粗 `no_grad()`、`time.monotonic()`、`phase (相位)`、`DAG (有向无环图)` 等专业名词，并在演讲时咬重发音。
2. **“拉齐技术语境”策略：**
   - **成员一**：不要只说“我们用了模型”，要说“我们这部分代码通过 `Torch No-Grad` **直接卸载了梯度计算图的开销，从而利用 CUDA 架构完成了硬件级别的边缘推理加速**”。
   - **成员二**：可以指着 `AT指令` 说：“市面上很多是调 API 拿数据，而我们这段逻辑是**利用时序算法直接向单片机底层下发控制帧指令，实现信道频率的即时复跳。**”
   - **成员三**：强调心电图**不是一张现成的GIF动画**，而是通过“纯数学分段三角函数（展现数学功底）+ 浏览器 GPU 渲染管道（展现前端性能深度）”实时画出来的。
   - **成员四**：强调“这**不是简单的提示词问答**，而是一个基于状态机编排的‘带记忆、会调用工具’反应式系统框架（Reactive Agent）”。